Report On

DESIGN AND IMPLEMENTATION OF AN END-TO-END DEVOPS PIPELINE USING A MINI PROJECT

Submitted in partial fulfillment of the requirements of Experiment 10 for DevOps

in Semester V of Third Year Artificial Intelligence & Data Science

by

Name of Student (Roll No. _____)

Name of Student (Roll No. _____)

Under the guidance of

Faculty Guide Name

University of Mumbai

Vidyavardhini's College of Engineering & Technology

Department of Artificial Intelligence & Data Science

(A.Y. 2026-27)

Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

CERTIFICATE

This is to certify that the project entitled “Design and Implementation of an End-to-End DevOps Pipeline Using a Mini Project” is a bonafide work of:

Name of Student (Roll No. _____) and Name of Student (Roll No. _____), submitted to the University of Mumbai in partial fulfillment of the requirements of the DevOps Experiment 10.

_____

Guide

Abstract

This project demonstrates an end-to-end DevOps workflow through a Student Task Manager web application. The application is developed using Python and Flask and provides basic features to add tasks, view the task list, mark tasks as completed, and delete tasks. Git and GitHub are used for version control and remote source-code hosting. GitHub Actions is configured to install dependencies, run automated tests with pytest, and build a Docker image when code is pushed or a pull request is created. Docker packages the application and its runtime dependencies into a container that can be executed locally. The project report documents the proposed system, pipeline stages, implementation files, test plan, containerization process, and evidence required for evaluation. Actual test, workflow, and container results must be recorded after running the project on the student's machine; screenshots should be added before final submission.

Keywords: DevOps, Git, GitHub, GitHub Actions, Continuous Integration, pytest, Docker, Flask, Containerization.

Table of Contents

Abstract — i

1 Introduction — 1

2 Problem Statement — 2

3 Proposed System — 3

3.1 Block Diagram, Description and Working — 3

3.2 Module Description — 4

4 Implementation Result and Analysis — 5

4.1 Implementation Screenshots — 6

5 Code — 7

6 Conclusion — 10

References — 11

1. INTRODUCTION

DevOps is a set of practices that brings software development and operations closer together. It encourages automation, repeatable processes, frequent integration, and reliable delivery. A DevOps pipeline automates steps that would otherwise be performed manually, such as retrieving source code, installing dependencies, executing tests, and packaging an application.

This mini project uses a Student Task Manager as the application under development. The application is intentionally small so that the main focus remains on the pipeline. It is built with Python and Flask, stored in a GitHub repository, tested with pytest, and packaged using Docker. GitHub Actions connects these activities through a workflow file.

The pipeline begins when a developer pushes code or opens a pull request. GitHub Actions checks out the repository, sets up Python, installs development dependencies, runs the automated tests, and builds a Docker image. The image can then be run locally as a container, allowing the application to be opened in a browser.

1.1 Objectives

• To manage project source code using Git and GitHub.

• To configure continuous integration with GitHub Actions.

• To automate application tests using pytest.

• To package the Flask application into a Docker image.

• To run and verify the application in a Docker container.

• To document the implementation and results for evaluation.

1.2 Scope

The scope includes source control, automated tests, a CI workflow, Docker image creation, and local container execution. External cloud deployment and publishing the image to a registry are not included in the current workflow. The application stores tasks in memory, so tasks reset when the server restarts.

2. PROBLEM STATEMENT

Small software projects often rely on manual steps to check code, install dependencies, run tests, and prepare an application for execution. These steps may be repeated inconsistently and errors can be missed if tests are not run after changes. A basic automated pipeline is needed to demonstrate how source control, testing, and containerization can be connected.

The problem addressed by this project is to design a simple web application and use it to demonstrate an end-to-end DevOps pipeline. The system should store source code in GitHub, automatically run tests on code changes, build a Docker image when tests pass, and provide a repeatable way to run the application.

2.1 Expected Outcome

The expected outcome is a working Student Task Manager, a GitHub repository containing source and configuration files, a GitHub Actions workflow, automated test cases, a Docker image, and a report with screenshots and verified results.

3. PROPOSED SYSTEM

The proposed system uses a lightweight Flask application and a CI workflow defined in YAML. Git records local source changes, GitHub stores the repository, and GitHub Actions runs the pipeline. pytest checks the main routes and task operations. Docker creates a self-contained image that can be run locally.

3.1 Block Diagram, Its Description and Working

Developer / VS Code

↓

Git Commit and Push

↓

GitHub Repository

↓

GitHub Actions: Install Dependencies → Run pytest

↓ (if tests pass)

Docker Image Build

↓

Run Container → Browser Verification

Working: The developer modifies the application and commits the changes with Git. After pushing to GitHub, the configured workflow starts automatically for the main or master branch and pull requests. The workflow installs dependencies and executes the automated test suite. If tests pass, Docker builds an image from the Dockerfile. The image can be started as a container and the application can be accessed through port 5000.

3.2 Module Description

3.2.1 Application Module

The Flask application is implemented in app.py. It provides a home page, an add-task route, a toggle-completion route, a delete-task route, and a health endpoint. The interface is rendered using templates/index.html.

3.2.2 Version Control Module

Git is used to track changes locally. GitHub stores the remote repository so that the code and pipeline configuration are available for review and evaluation.

3.2.3 Automated Testing Module

The test_app.py file uses pytest and Flask's test client. It checks the home page, task creation, rejection of blank tasks, task completion, task deletion, and the health endpoint.

3.2.4 Continuous Integration Module

The .github/workflows/ci.yml workflow runs when code is pushed or a pull request targets main or master. It checks out the code, sets up Python 3.12, installs requirements-dev.txt, runs pytest, and builds the Docker image.

3.2.5 Containerization Module

The Dockerfile uses python:3.12-slim as a base image, installs Flask, copies the application files, exposes port 5000, and runs the application as a non-root user. The image can be run with a port mapping from host port 5000 to container port 5000.

4. IMPLEMENTATION RESULT AND ANALYSIS

This section describes how the project should be run and how results should be recorded. The final report must reflect the actual outputs obtained on the student's machine. Do not mark a stage as successful until it has been executed and verified.

4.1 Local Application Setup

Install Python and open the extracted project folder in VS Code. Create a virtual environment, install the development requirements, and start the Flask application using the commands below.

```
python -m venv .venv
.venv\\Scripts\\Activate.ps1
pip install -r requirements-dev.txt
python app.py
```

Open http://127.0.0.1:5000 in a browser. Add a task, mark it complete, and delete it to verify the basic interface.

4.2 Automated Testing

```
pytest -v
```

The test suite contains six tests. It checks the home page, adding a task, blank-task validation, toggling completion, deleting a task, and the health endpoint. Record the actual pass/fail count after running the command.

Actual test result: _____

4.3 GitHub Actions Pipeline

Create a GitHub repository and push the project. Open the Actions tab to inspect the workflow run. A successful run should show the test step passing and the Docker image build step completing.

Actual workflow status and run date: _____

4.4 Docker Build and Container Execution

```
docker build -t student-task-manager .
docker run --rm -p 5000:5000 student-task-manager
```

With the container running, visit http://localhost:5000. The endpoint http://localhost:5000/health should return a JSON response indicating that the service is healthy.

Actual Docker build and run result: _____

4.5 Analysis

The workflow connects code changes to automated verification and image creation. Automated tests can detect regressions in the application's main routes. Docker provides a repeatable runtime package. In this implementation, the CI pipeline builds the image but does not push it to a registry or deploy it to a cloud platform. The final evaluation should clearly distinguish the verified stages from stages not performed.

4.6 IMPLEMENTATION SCREENSHOTS

Insert screenshots captured from your own system in the spaces below. These screenshots provide evidence that the project and pipeline were actually executed.

Figure 1: Student Task Manager running in the browser

[Insert screenshot here]

Figure 2: Automated pytest results

[Insert screenshot here]

Figure 3: GitHub repository showing project files

[Insert screenshot here]

Figure 4: GitHub Actions workflow successful run

[Insert screenshot here]

Figure 5: Docker image build output

[Insert screenshot here]

Figure 6: Running Docker container and application

[Insert screenshot here]

5. CODE

The full source code and configuration files are included in the project ZIP. The following excerpts illustrate the key parts of the implementation.

5.1 Flask Application — app.py

```
@app.get('/')
def index():
    return render_template('index.html', tasks=tasks)

@app.post('/add')
def add_task():
    global next_id
    title = request.form.get('title', '').strip()
    if title:
        tasks.append({'id': next_id, 'title': title, 'done': False})
        next_id += 1
    return redirect(url_for('index'))
```

5.2 Health Endpoint

```
@app.get('/health')
def health():
    return {'status': 'ok'}, 200
```

5.3 GitHub Actions Workflow — ci.yml

```
name: Student Task Manager CI
on:
  push:
    branches: ['main', 'master']
  pull_request:
    branches: ['main', 'master']
  workflow_dispatch:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install -r requirements-dev.txt
      - run: pytest -v
      - run: docker build -t student-task-manager:${{ github.sha }} .
```

5.4 Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
COPY templates ./templates
EXPOSE 5000
CMD ['python', 'app.py']
```

Note: Refer to the full files in the repository for the complete code. The excerpts above are for report readability.

6. CONCLUSION

The Student Task Manager mini project is designed to demonstrate core DevOps practices through a small web application. It combines Git and GitHub for version control, pytest for automated testing, GitHub Actions for continuous integration, and Docker for containerization. Running the workflow after pushing the repository makes the testing and image-building steps repeatable and visible.

The project is complete for evaluation only after the application, tests, workflow, and Docker container have been run and verified. The actual outcomes and screenshots must be added to this report. The current implementation demonstrates local container execution; automated registry publishing and public cloud deployment are outside the configured scope.

REFERENCES

[1] Git Documentation, “Git Reference Manual.” [Online]. Available: https://git-scm.com/doc

[2] GitHub, “GitHub Actions Documentation.” [Online]. Available: https://docs.github.com/actions

[3] Docker, “Docker Documentation.” [Online]. Available: https://docs.docker.com/

[4] Pallets Projects, “Flask Documentation.” [Online]. Available: https://flask.palletsprojects.com/

[5] pytest Development Team, “pytest Documentation.” [Online]. Available: https://docs.pytest.org/